Date

Expt. No. 8. Page No. 26.

Aim :- To write a C program to count the occurrence of each character in a file.

Algorithm:-

1. Start
2. Open file
3. Initialize array of size 256 with zero
4. Read characters one by one
5. Increment corresponding array index.
6. Repeat until EOF
7. Print frequencies
8. Close files
9. Stop.

Program

```
#include <stdio.h>
int main()
{
 FILE *fp;
 int freq[256] = {0};
 char ch;
 fp = fopen("input.txt", "r");
 if(fp == NULL)
{
 printf("Error opening file!\n");
 return 1;
}
 while ((ch = fgetc(fp)) != EOF)
{
```

Teacher's Signature

```
    }
    freq[(unsigned char) ch]++;
}
printf("Character Frequencies:\n");
for (int i=0; i<256; i++)
{
  if(freq[i]>0)
{
  printf("%c = %d\n", i, freq[i]);
}
  }
fclose(fp);
return 0;
}
```

Viva Question

Q1 How do you define an identifier in your program?

Ans An identifier is a name given to variables, functions, arrays or other program elements. The program identifies it as a sequence of letters, digits & underscores that does not start with a digit.

Q2 How do you distinguish identifiers from keywords?

Ans The program compares each word with a predefined list of keywords. If the word matches a keyword, it is classified as a keyword; otherwise it is treated as an identifier if it follows the rules for naming identifiers.

Teacher's Signature

Date

Expt. No. Page No. 28.

Q3 How do you avoid counting identifiers inside comments & strings?

Ans The program detects single-line & multi-line comments & ignores their contents. It also tracks string literals & skips the text inside quotes to ensure that only valid identifiers are counted.

Q4 How does your program handle case sensitivity in identifiers?

Ans The program treats uppercase & lowercase letters as different characters, for example name, Name & NAME are considered three different identifiers in C & C++.

Q5 What is significance of underscore character in identifiers?

Ans An underscore (_) is used in identifiers to separate words & improve readability. It can appear at the beginning or within an identifier for example, _count or total_marks.

Teacher's Signature

Date

Expt. No. 9. Page No. 29.

Aim :- To implement a symbol table using C.

Algorithm :-

1. Start
2. Define structure for symbol table
3. Insert entries
4. Store name & type
5. Display table
6. Stop

Program (C) :-

```
#include <stdio.h>
#include <string.h>
  struct symbol
    {
     char name[20];
     char type[10];
    };
struct symbol table[100];
int count = 0;
// insert symbol
   void insert(char name[], char type[])
{
 void insert(char name[
strcpy(table[count].name, name);
strcpy(table[count].type, type);
  count++;
}
```

Teacher's Signature

```
// Display table
void table
void display()
    printf("\n Symbol Table:\n");
    printf(" None \t Type \n");
    for(int i=0; i< count; i++){
        printf("%s \t %s \n", table[i].name,
                               table[i].type);
    }
}

int main(){
    insert("a","int");
    insert("b", "float");
    insert("sum", "function");
    display();
    return 0;
}
```

Viva Question:-

Q1 What are the primary responsibilities of a lexical analyzer?

Ans A lexical analyzer reads the source code & converts it into tokens such as keywords, identifiers, operators & comments & detects invalid characters or lexical errors.

Teacher's Signature

Date

Expt. No. Page No. 31

Q2 How does the lexical Analyzer communicate with the parser?

Ans The lexical analyzer communication with the parser by sending tokens one by one. Each token contains information about its type & sometimes its value, which helps the parser check the syntax of the program.

Q3 What is the input & output of your lexical analyzer?

Ans The input of a lexical analyzer is the source code written in a programming language. Its output is a sequence of tokens, such as keywords, identifiers, operators & constants which are passed to the parser.

Q4 How do you implement the state transitions in your manual lexical analyzer?

I implement state transitions using conditional statement & loops to checks each character of the source code. Based on the character, the analyzer changes states to identify tokens such as identifiers, numbers, operators & string.

Q5 How do you handle lexical errors like invalid characters?

Ans The lexical analyzer detects characters that do not belong to the programming language's valid tokens, it displays an error msg, often with the line number & continues scanning the remaining source code whenever possible.

Teacher's Signature

Date

Expt. No. 10 Page No. 22

Aim:- To write a YACC program to recognise string of the form $a^n b^n$.

Algorithm:-

1. Start
2. Define grammar rule
3. Use LEX to generate tokens
4. Parse input using YACC.
5. If grammar matches → valid.
6. Else → invalid
7. Stop.

Program (c)

LEX file

```
%{
#include "y.tab.h"
%}
%%
a {return A;}
b {return B;}
\n {return 0;}
%%
int yywrap()
{return 1;}
```

YACC file

```
%{
#include<stdio.h>
int yylex();
void yyerror(const char *s);
%}
```

Teacher's Signature

```
%token AB
%%
S : ASB
  | AB
  ;
%%

int main()
{
printf("Enter string :\n");
yyparse();
printf("Valid string:\n");
return 0;
}

void yyerror(const char *s)
{
printf("Invalid string\n");
}
```

Viva Questions

Q1 How does your C program differ from the LEX-based implementation?

Ans My C program uses manually written logic & conditions to identify tokens, while LEX uses regular expressions & predefined rules. LEX automatically generates a lexical analyzer, making the implementation easier & faster.

Teacher's Signature

Date

Expt. No. Page No. 33

Q2 What logic do you use to identify word boundaries?
I use delimiters such as spaces, tabs, newlines, & special symbols to identify word boundaries. When a delimiters is encountered, the current word is processed as a complete token.

Q3 How do you handle different type of whitespaces?
I use function like isspace() in C to detect white space character such as spaces, tabs, & newlines. These characters are ignored or treated as seperators while processing tokens.

Q4 What is the importance of EOF (End of File) marker in your program.
EOF indicates that no more input is available to read from the file. It helps the program stop scanning safely & process any remaining characters or tokens before termination.

Q5 How do you ensure your program handles empty file correctly?
I check whether the file contains any data before processing it. If the file is empty, the program displays an appropriate messeye & terminates safely without causing errors.

Teacher's Signature